OPERATING SYSTEMS

ORANGE ASSIGNMENT-2

NAME:DHANUSH S SHEKHAR
SRN:PES1UG24CS662
SECTION:K

Phase 1:

Screenshot 1A:

```
dhanush-s-shekhar@dhanush-s-shekhar-QEMU-Virtual-Machine:~/PES1UG24CS662-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B:

```
dhanush-s-shekhar@dhanush-s-shekhar-QEMU-Virtual-Machine:~/PES1UG24CS662-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

Phase 2:

Screenshot 2A:

```
dhanush-s-shekhar@dhanush-s-shekhar-QEMU-Virtual-Machine:~/PES1UG24CS662-pes-vcs$ make clean
make test_tree
./test_tree
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o test_tree test_tree.o object.o tree.o index.o -lcrypto
Serialized tree: 253 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B:

```
dhanush-s-shekhar@dhanush-s-shekhar-QEMU-Virtual-Machine:~/PES1UG24CS662-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
dhanush-s-shekhar@dhanush-s-shekhar-QEMU-Virtual-Machine:~/PES1UG24CS662-pes-vcs$ xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e
650eda471e566fcf
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
```

Phase 3:

Screenshot 3A:

```
dhanush-s-shekhar@dhanush-s-shekhar-QEMU-Virtual-Machine:~/PES1UG24CS662-pes-vcs$ ulimit -s unlimited
dhanush-s-shekhar@dhanush-s-shekhar-QEMU-Virtual-Machine:~/PES1UG24CS662-pes-vcs$ rm -rf .pes
ulimit -s unlimited
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Untracked files:
  untracked:  test_sequence.sh
  untracked:  tree.c
  untracked:  object.c
  untracked:  b.txt
  untracked:  index.c
  untracked:  README.md
  untracked:  a.txt
  untracked:  test_tree.c
  untracked:  gen_tree.c
  untracked:  Makefile
  untracked:  tree.h
  untracked:  pes.h
  untracked:  commit.h
  untracked:  phase3.txt
  untracked:  pes.c
  untracked:  commit.c
  untracked:  index.h
  untracked:  test_objects.c
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776336275 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776336275 6 file2.txt
```

Screenshot 3B:

```
dhanush-s-shekhar@dhanush-s-shekhar-QEMU-Virtual-Machine:~/PES1UG24CS662-pes-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776336275 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776336275 6 file2.txt
```

Phase 4:

Screenshot 4A:

```
dhanush-s-shekhar@dhanush-s-shekhar-QEMU-Virtual-Machine:~/PES1UG24CS662-pes-vcs$ rm -rf .pes
./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"
echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"
echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
./pes log
Initialized empty PES repository in .pes/
Committed: 833d99dd4db2... Initial commit
Committed: 353a3fadb2e5... Add world
Committed: 13622eb1dfaa... Add farewell
commit 13622eb1dfaa78c63dc49eb009ed85025402e71c6a4827ea3427feb84fc73dfa
Author: PES User <pes@localhost>
Date:   1776338490

    Add farewell

commit 353a3fadb2e50b9c8a7e75ab4db966831871c806d2cad21b8c2c712b04f5da62
Author: PES User <pes@localhost>
Date:   1776338490

    Add worldell

commit 833d99dd4db29dc6c7120bde4cfe38ccbaad383ff22063cbd85ab38b2c9cae7c
Author: PES User <pes@localhost>
Date:   1776338490

    Initial commit
```

Screenshot 4B:

```
dhanush-s-shekhar@dhanush-s-shekhar-QEMU-Virtual-Machine:~/PES1UG24CS662-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/13/622eb1dfaa78c63dc49eb009ed85025402e71c6a4827ea3427feb84fc73dfa
.pes/objects/35/3a3fadb2e50b9c8a7e75ab4db966831871c806d2cad21b8c2c712b04f5da62
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7c/ecb2ed16b76f5db0f2dbb9e71ac2e1807e5f9855abf5ff15c55d8544a747a2
.pes/objects/83/3d99dd4db29dc6c7120bde4cfe38ccbaad383ff22063cbd85ab38b2c9cae7c
.pes/objects/91/e9e7eadeac6f0fab92fc7af19e897082baa535c930404b33aa53953e7019c4
.pes/objects/cf/30457f979bec20cec63b552296c3f7d88ca4e934085b87fb45d4bf4c6dfd2b
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

Screenshot 4C:

```
dhanush-s-shekhar@dhanush-s-shekhar-QEMU-Virtual-Machine:~/PES1UG24CS662-pes-vcs$ cat .pes/refs/heads/main
13622eb1dfaa78c63dc49eb009ed85025402e71c6a4827ea3427feb84fc73dfa
dhanush-s-shekhar@dhanush-s-shekhar-QEMU-Virtual-Machine:~/PES1UG24CS662-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Phase 5:

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout`

`<branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

To implement pes checkout <branch>: you'd update .pes/HEAD to point to the new branch, read that branch's commit, walk its tree, and overwrite the working directory files to match. The complexity is handling conflicts with modified tracked files.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Detecting a "dirty" conflict: for each file that differs between branches, check if the index entry's hash matches the working file's hash. If they differ, the file is locally modified — refuse checkout.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

In detached HEAD state, new commits have no branch pointing to them. They're reachable only via the commit hash shown in HEAD. To recover, create a branch pointing to that hash: git branch recovery-branch <hash>.

Phase 6:

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

GC algorithm: start from all branch refs, walk each commit's tree recursively, collect all reachable hashes into a set. Then list every file in .pes/objects/, delete any whose hash isn't in the set. Use a hash set (like a C uthash or a boolean array over hashes) for O(1) lookup. For 100k commits × ~3–5 objects each ≈ 300k–500k objects to visit.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Race condition: GC marks an object as unreachable (it's not yet referenced), then a concurrent commit writes a new commit that references that object, then GC deletes it before the commit finishes writing. Git avoids this by using a grace period (objects newer than ~2 weeks are never collected, giving in-flight operations time to complete)